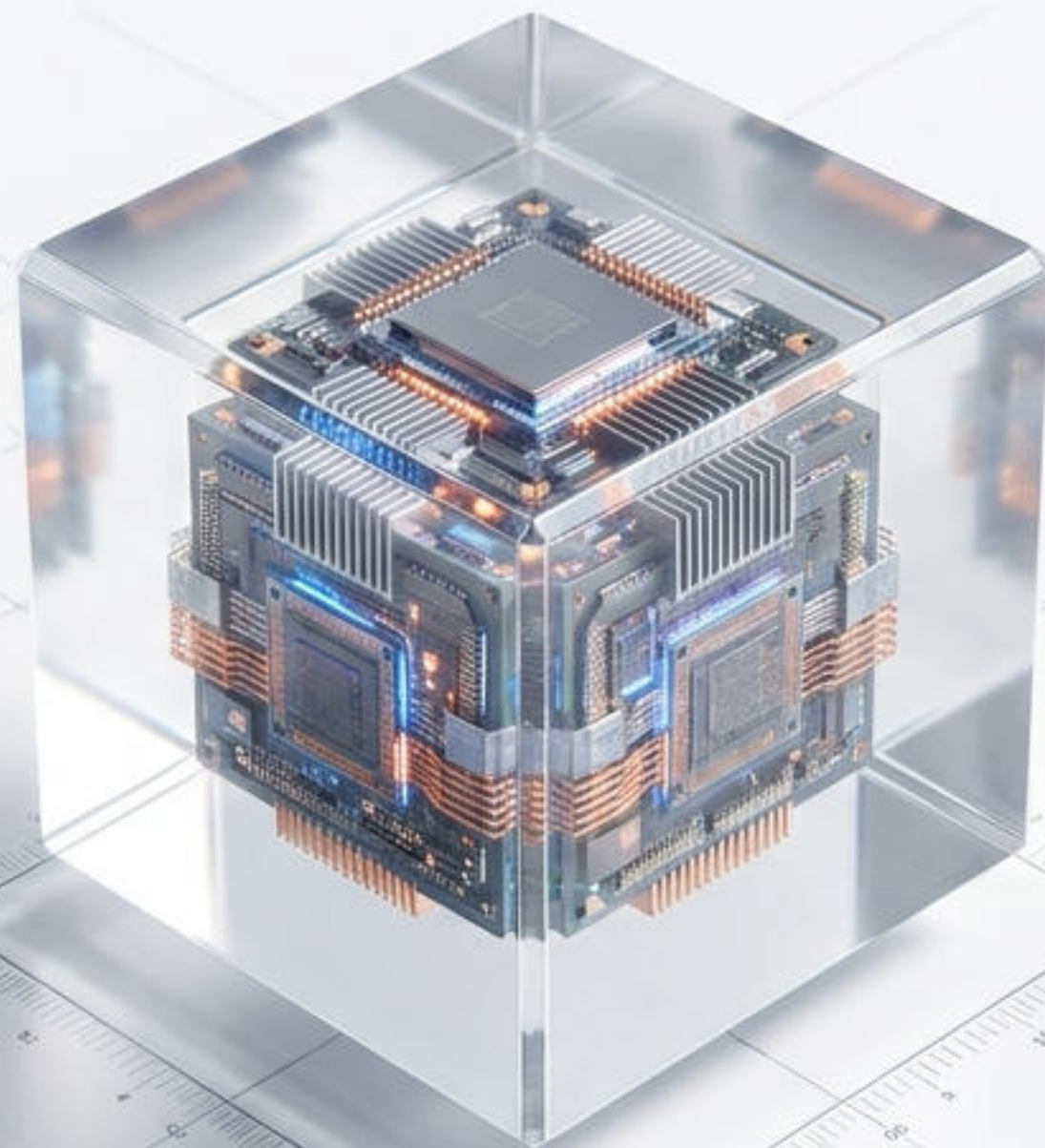


[SYS.INIT]
[VOL. 01]



Capítulo 1 — Pilares de la POO

Sesión 1: Abstracción de Datos y Encapsulación

**Años 60:
Simula**

El origen lógico

**Años 70:
Smalltalk**

El paradigma puro

**Años 80/90:
C++ / Java**

La estandarización

Dominando la Complejidad

El Problema Histórico



Gestionar sistemas de software masivos e inestables con un estado global compartido y caótico.

La Solución Fundacional



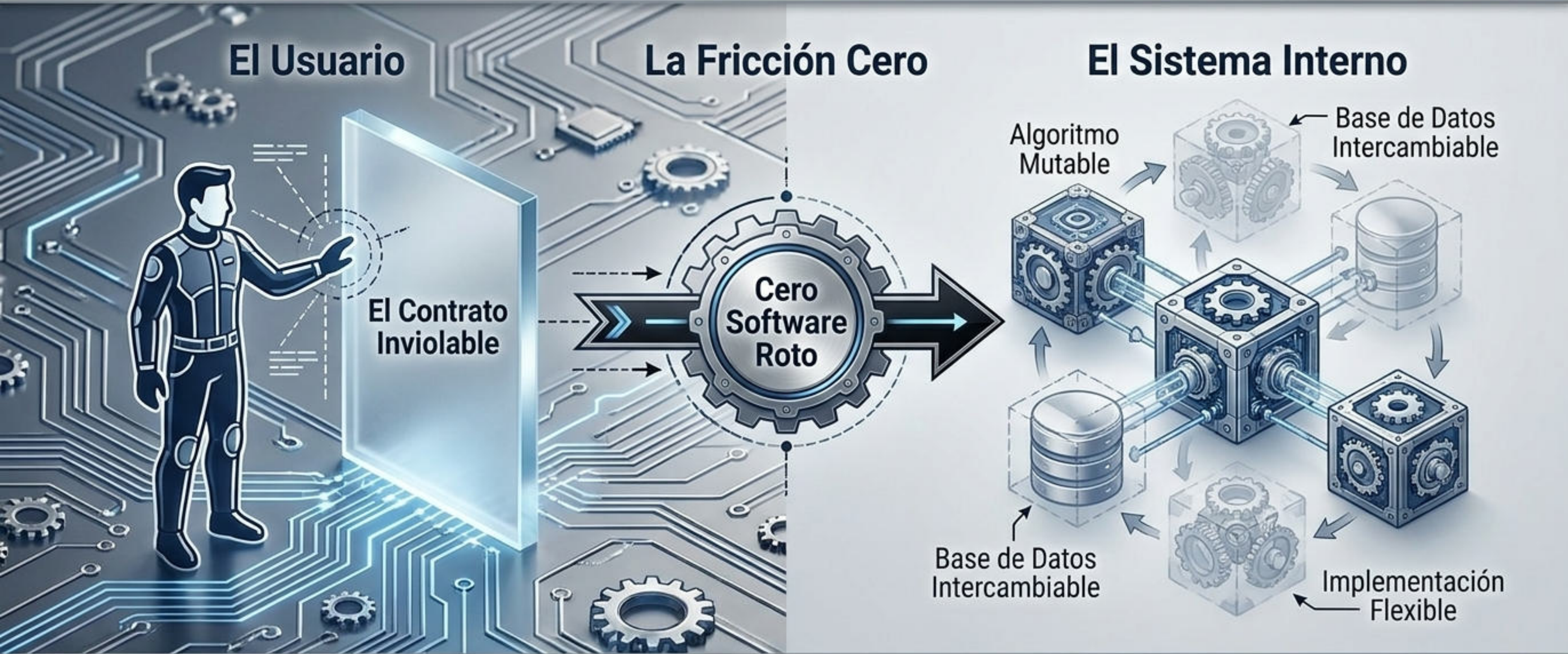
Abstraer la realidad en "Objetos" independientes que se comunican exclusivamente enviándose mensajes.

Pilar 1: Abstracción de Datos

1. **El Panel de Control:** Asocia un tipo de dato subyacente exclusivamente con las operaciones que lo manipulan.
2. **La Regla de Ignorancia:** El usuario interactúa con la interfaz operativa; la representación interna de la maquinaria es irrelevante y no accesible.



El Poder de la Abstracción: La Invariabilidad

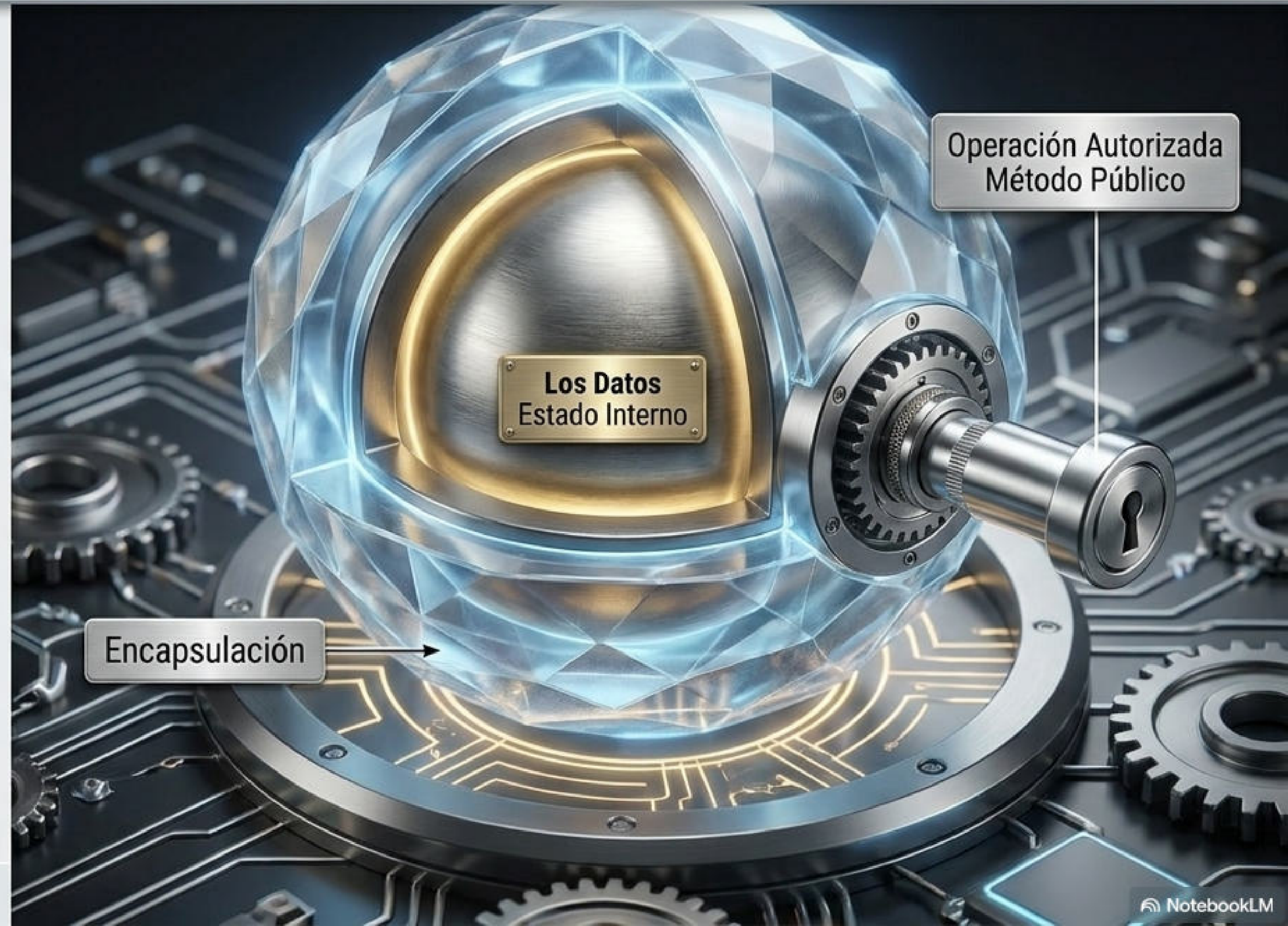


La separación **garantiza** que el mecanismo subyacente puede mutar sin romper el código de quien lo utiliza.

Pilar 2: Encapsulación

El Campo de Fuerza

- La fusión física y lógica de los datos con sus operaciones.
- El estado interno del objeto queda oculto y protegido bajo estrictas reglas de acceso.
- Es el fin definitivo de la manipulación accidental.



Matriz Diagnóstica de Separación de Responsabilidades

Desambiguación de conceptos frecuentemente confundidos

	Abstracción (La Interfaz)	Encapsulación (El Núcleo)
Propósito Principal	Diseño y simplificación lógica.	Seguridad e integridad de la implementación.
Problema que resuelve	Sobrecarga cognitiva y complejidad mental.	Corrupción de datos y estados inconsistentes.
Pregunta Clave	¿Qué hace este objeto?	¿Cómo protejo el estado de este objeto?

El Peligro del Estado Interno Expuesto

Step 1: Imagina un objeto Point.

Datos: Coordenada X, Coordenada Y, Distancia al origen.

Step 2: El Desastre.

Si el usuario modifica la Coordenada X de forma directa y olvida recalcular la Distancia, el estado interno se corrompe.

Los datos mienten. El sistema colapsa.



Tutorial Paso 1: Definir la Abstracción

La Promesa

⚙️ [COMMAND] setX()

⚙️ [COMMAND] setY()

🔍 [QUERY] x()

🔍 [QUERY] y()

🔍 [QUERY] DistanceFromOrigin()

Diseñamos exclusivamente las acciones públicas. Al usuario solo le entregamos los botones de control; ocultamos la matemática subyacente.

Tutorial Paso 2: Aplicar Encapsulación

El Candado



```
private double x;
```

```
private double y;
```

```
private double distance;
```

Levantando el Campo de Fuerza

- Usamos **modificadores de acceso** (**private** o **protected**). Al declarar las variables como privadas, bloqueamos permanentemente la manipulación directa desde el exterior.
- El dato ahora es inviolable.

Tutorial Paso 3: Operaciones Autorizadas

El Guardián

```
public void setX(double x) {
```

setX(double x)



Dato del Usuario (x)

```
setY()  
setX.Internal = x y;  
distanceF y;
```

Estado Interno
Actualizado



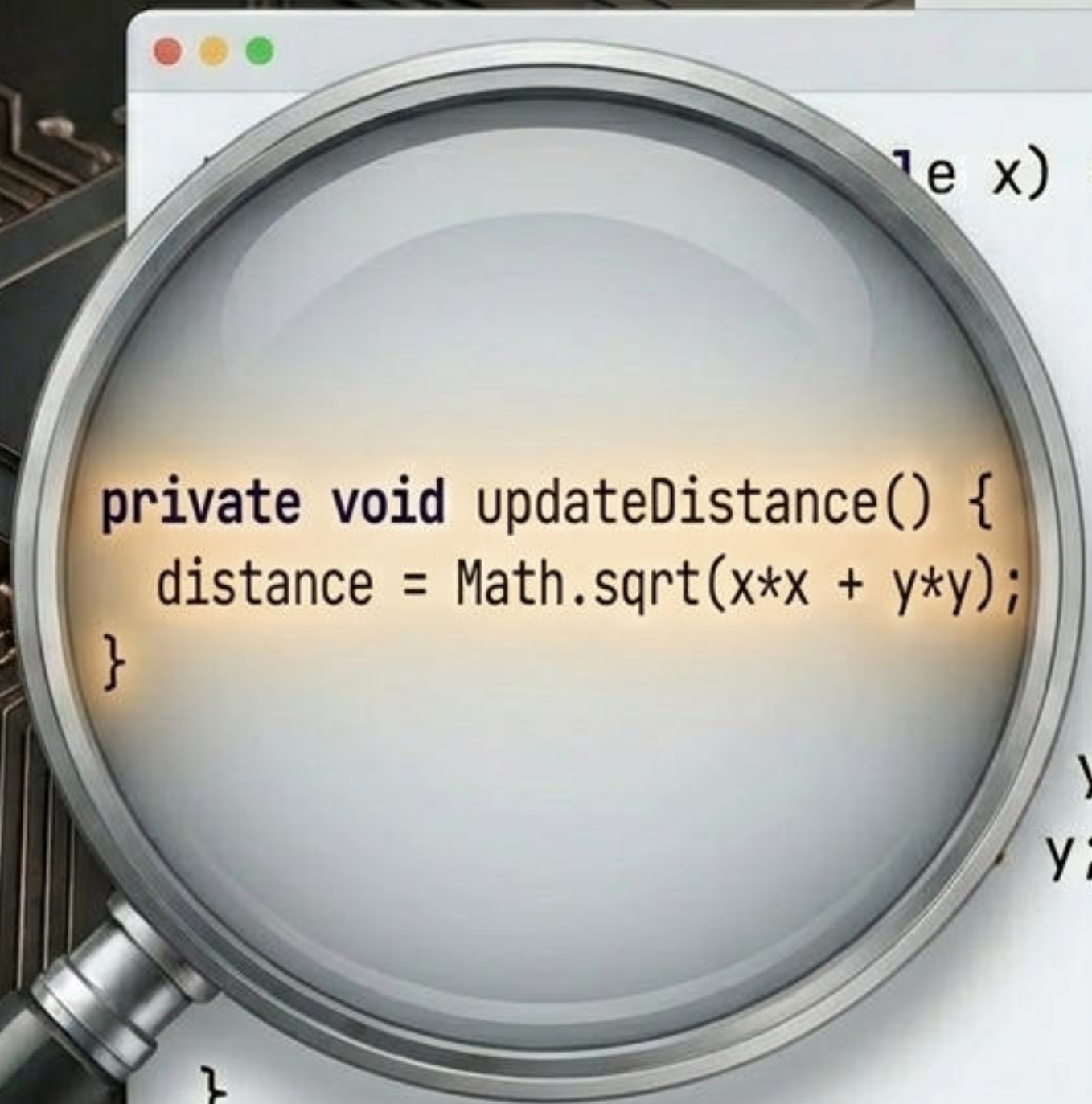
```
}
```

El método público recibe el dato. Al interior de este canal seguro, el objeto asume el control total de la validación y el flujo interno.

El usuario delega la responsabilidad estructural al objeto.

Tutorial Paso 4: Manteniendo la Integridad

El Secreto Interno



```
private void updateDistance() {  
    distance = Math.sqrt(x*x + y*y);  
}
```

```
y;  
y;
```

La Magia Automática

- Cada vez que se ejecuta `setX()` o `setY()`, el sistema invoca internamente a `updateDistance()`.
- El usuario ignora que esto sucede, pero el objeto garantiza de forma matemática que su estado jamás será inconsistente. Problema resuelto.

Arquitectura del Diseño por Contrato

Garantiza que el objeto jamás colapsará ni generará datos corruptos.

Se compromete a utilizar exclusivamente los métodos públicos permitidos.

El Productor
(Clase)

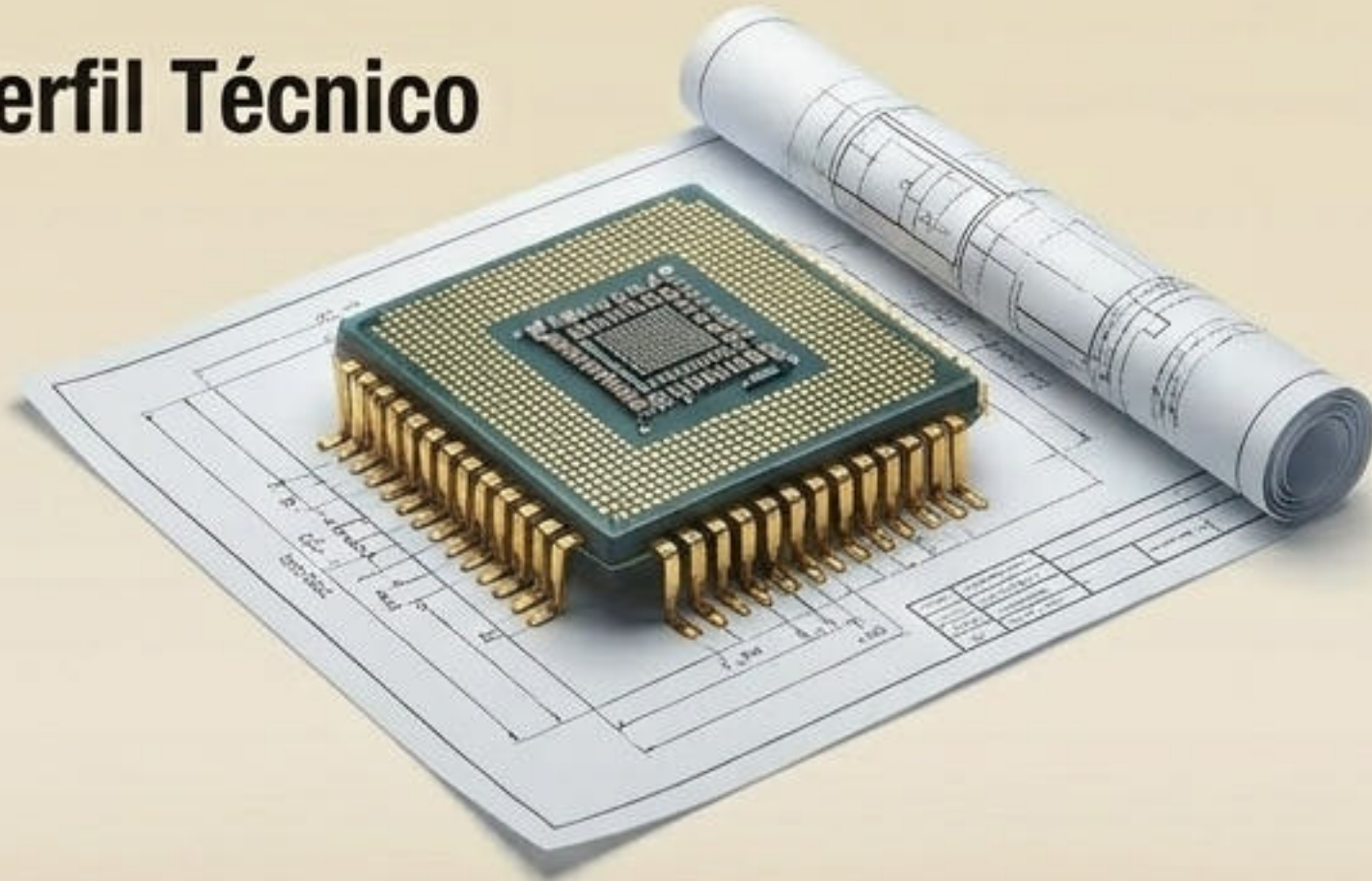
El Contrato
Estricto

El Consumidor
(Usuario)

Abstracción + Encapsulación = Responsabilidad mutua y sistemas impenetrables.

El Impacto Dual: Ingeniería y Negocio

Perfil Técnico



- **Aislamiento Quirúrgico:** Permite encapsular y aislar bugs sin afectar otras áreas.
- **Reutilización:** Código inmensamente modular y transportable.
- **Prevención de Cascada:** Si algo falla, se rompe solo por dentro, protegiendo el sistema global.

Perfil Gerencial



- **Reducción de Costos:** Mantenimiento técnico drásticamente más económico.
- **Desarrollo Paralelo:** Múltiples equipos pueden trabajar sin fricción ni bloqueos.
- **Escalabilidad:** Sistemas a prueba de errores humanos futuros, protegiendo la inversión.

La Anatomía de la Perfección Estructural

Capa 1: Los Datos
El Estado Interno

Capa 2: La Encapsulación
La Privacidad

Capa 1: Los Datos
El Estado Interno

Capa 3: La Abstracción
La Interfaz Pública

El ecosistema de la P00 perfecto: Ocultar lo vital. Exponer lo estrictamente necesario. Garantizar la **integridad** matemática siempre.

Has dominado la anatomía del Objeto.



Pero un objeto aislado no genera un sistema.
¿Cómo se comunican y orquestan entre sí para resolver problemas a escala masiva?

Próxima Sesión: Objetos, Mensajes y Métodos.